

Option C Task Report

COS30018 – Intelligent Systems

Lecturer: Dr. Nguyen Manh Toan

Group: 3.1 – OpenBuffer

Task name: **C.3 – Data Processing 2**



In Task C3, we've implemented 2 main functions for candlestick and boxplot plotting. To support them, we've also code other helper functions that let user input their desired "n" days aggregation.

Here is the breakdown of what we've done for each requirement:

I. Candlestick Chart Function:

```
def plot_candlestick_chart(  
    df: pd.DataFrame,  
    n_days: int = 1,  
    title: str = 'Candlestick Chart',  
    show: bool = True  
):
```

1) Params

- **df** (pd.DataFrame): The input dataset containing the stock price history. For the chart to render properly, this DataFrame must have a DatetimeIndex (dates as row indices) and include at least four columns: 'Open', 'High', 'Low', and 'Close'.
- **n_days** (int, default=1): The aggregation factor specifying how many trading days a single candlestick represents. If set to 1, each candle reflects a single day. If set to , every block of trading days is squeezed into one candle to clean up short-term noise.
- **title** (str, default='Candlestick Chart'): The primary heading text displayed at the top of the generated visualization window.
- **show** (bool, default=True): A flag determining whether the function should immediately trigger plt.show() to pop up a separate graphic window on your screen.

2) How it works

2.1. Input Validation

```
if n_days < 1:  
    raise ValueError("n_days must be >= 1.")  
  
# mplfinance needs these exact column names and a DatetimeIndex - check now  
# so the error is clear instead of coming from inside the library.  
required = {'Open', 'High', 'Low', 'Close'}  
if not required.issubset(set(df.columns)):  
    raise ValueError(f"DataFrame must contain columns {required}; got {list(df.columns)}.")  
if not isinstance(df.index, pd.DatetimeIndex):  
    raise ValueError("DataFrame index must be a DatetimeIndex for a candlestick chart.")
```

Before plotting, the code validates that **n_days** is a valid positive number. It then checks to ensure all critical financial columns exist and that the index consists of proper timestamps. This explicitly prevents runtime crashes deeper inside third-party plotting routines.

2.2. Data Aggregation

```
plot_df = _aggregate_n_days(df, n_days)      # no-op when n_days == 1
has_volume = 'Volume' in plot_df.columns      # only show volume if we have it
```

The data is passed to the internal helper **_aggregate_n_days()**. If **n_days** is 1, it returns the original dataframe unchanged. If **n_days** is greater than 1, it groups every consecutive rows via floor division (//) and applies financial pooling rules (will later be explained in the helper function description).

The program then checks if a **'Volume'** column is present in the dataset. This boolean flag determines whether to add a secondary bar chart panel for volume under the primary price panels.

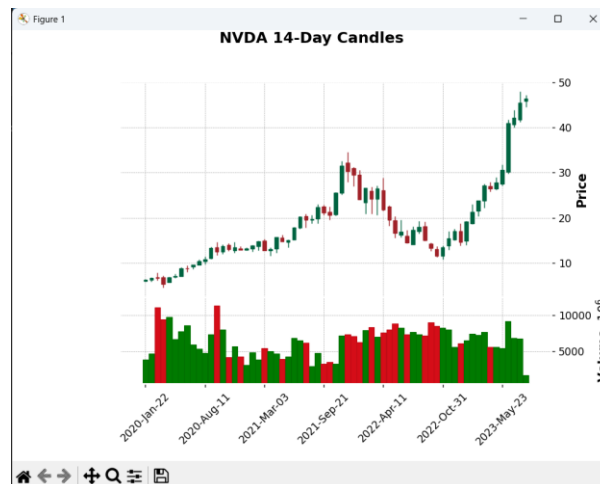
2.3. Plotting

```
mpf.plot(plot_df, type='candle', style='charles', volume=has_volume,
          title=chart_title, ylabel='Price')

if show:
    plt.show()
```

Finally, we used **mplfinance** library to plot out the candelsticks via **mpf.plot()**.

The final result with **n=14** or a candle represents 2 weeks of data:



II. Boxplot Chart Function

```
def plot_boxplot_chart(  
    df: pd.DataFrame,  
    column: str = 'Close',  
    n_days: int = 20,  
    step: int | None = None,  
    title: str = 'Boxplot Chart',  
    show: bool = True  
):
```

1) Params

- **df** (pd.DataFrame): The input historical stock dataset. It must contain the specific numeric column you want to analyze and use a DatetimeIndex to label the timelines.
- **column** (str, default='Close'): The name of the specific data column to analyze. This targets which metric (typically 'Close', but could be 'Open' or 'Volume') will have its values extracted to calculate statistical distributions.
- **n_days** (int, default=20): The window size. It determines how many consecutive trading days of data are captured inside each individual box plot.
- **step** (int | None, default=None): The stride or movement rate of the window.
 - If step is None, it automatically falls back to equal n_days, meaning adjacent windows sit back-to-back without overlapping.
 - If step is set to 1, the window advances by a single day at a time, creating a continuous sliding window with high data overlap.
- **title** (str, default='Boxplot Chart'): The main title text rendered at the top of the figure.
- **show** (bool, default=True): A flag specifying whether to immediately display the graphic window using plt.show(). If False, it closes the figure object in the background to free up system memory.

2) How it works

2.1. Input Validation and Fallback Setup

```
if column not in df.columns:  
    raise ValueError(f"Column '{column}' not found in DataFrame; got {list(df.columns)}.")  
if n_days < 1:  
    raise ValueError("n_days must be >= 1.")  
if step is None:  
    step = n_days
```

The code ensures the requested data column actually exists in the DataFrame and confirms **n_days** is a valid positive window size. It then sets up the default non-overlapping behavior if no explicit step value is supplied.

2.2. Window Slicing and Labeling

```
series = df[column]

# One window (= one box) per slide. Stop at len - n_days so the last window is still full length.
windows = []      # the n_days values for each box
labels = []       # x label = the last date in each window
for start in range(0, len(series) - n_days + 1, step):
    window = series.iloc[start:start + n_days]
    windows.append(window.values)
    labels.append(window.index[-1].strftime('%Y-%m-%d'))
```

The program loops through the data series by advancing the index pointer by the size of step at each iteration. It stops at **len(series) - n_days + 1** to guarantee that the final window contains a full set of elements.

- **windows.append(window.values)**: Extracts the raw array of numeric values for that block and saves it. This array becomes the data source used to calculate the box distribution.
- **labels.append(window.index[-1].strftime('%Y-%m-%d'))**: Grabs the very last date timestamp of that specific window, formats it as a clean string (YYYY-MM-DD), and saves it as the X-axis label for that box.

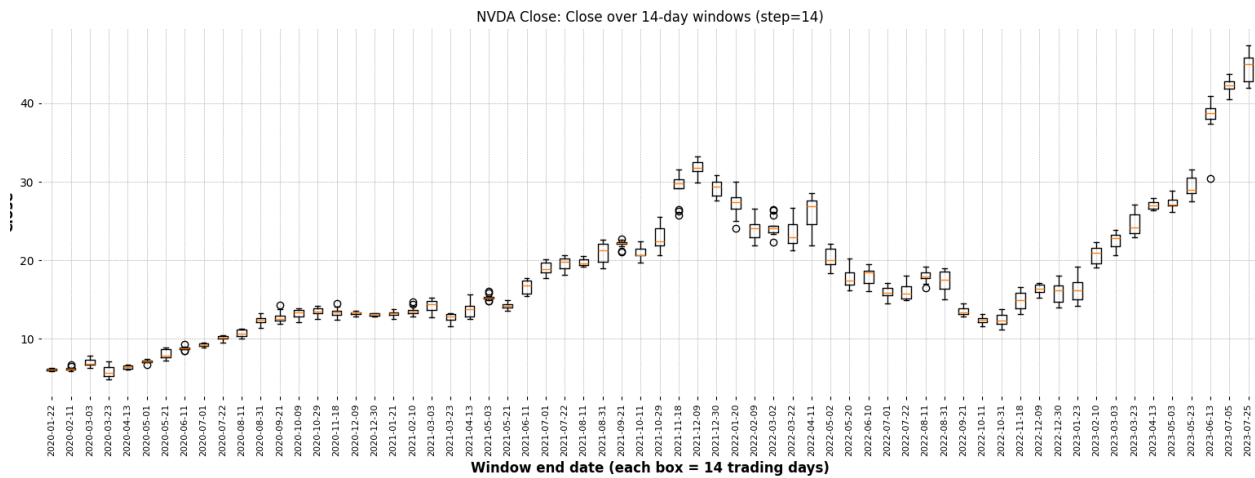
2.3. Plotting

```
fig, ax = plt.subplots(figsize=(max(8, len(windows) * 0.4), 6))
ax.boxplot(windows, labels=labels)
```

This code adjusts the layout width using **max(8, len(windows) * 0.4)** to make sure that if there are a massive number of boxes (when step=1), the chart expands horizontally so the elements do not squish together.

The **ax.boxplot(windows, labels=labels)** calculates the distribution metrics for each window array and draws the standard box components

The final result with n=14 or a candle represents 2 weeks of data:



III. Helper Functions

1. `_aggregate_n_days()`

```
def _aggregate_n_days(df: pd.DataFrame, n_days: int) -> pd.DataFrame:
```

`n_days (int)`: The target number of active trading days to compress into a single row. If $n \leq 1$, no data compression is required.

Instead of resampling via calendar dates (which introduces blank rows for non-trade days) this math trick groups rows entirely by their positions in the array.

An array of sequential integers from 0 to $\text{len}(\text{df}) - 1$ is created via `np.arange()`, then floor-divided (`//`) by `n_days`. For instance, if `n_days=3`, the positions generate group IDs: `[0, 0, 0, 1, 1, 1, 2, 2, 2...]`. Rows holding matching IDs are compressed together, guaranteeing every output candle contains exactly valid trading records.

Dynamic Financial Rule Assignment:

```
agg_rules = {}
for col in df.columns:
    low = col.lower()
    if low == 'open':
        agg_rules[col] = 'first'
    elif low == 'high':
        agg_rules[col] = 'max'
    elif low == 'low':
        agg_rules[col] = 'min'
    elif low == 'close':
        agg_rules[col] = 'last'
    elif low == 'volume':
        agg_rules[col] = 'sum'
    else:
        agg_rules[col] = 'last'
# safety fallback for any other column

agg = df.groupby(groups).agg(
    agg_rules)
```

The loop populates a Pandas aggregation dictionary by mapping case-insensitive column names to structural financial pooling math:

- **Open:** Extract the opening trade value from the very first day of the block ('first').
 - **High/Low:** Calculate the structural extremities over the multi-day span ('max' and 'min').
 - **Close:** Grab the final settling price at the end of the block ('last').
 - **Volume:** Sum all transaction activity over the block ('sum').
2. `_ask_int()`: Keep asking until it gets a whole number ≥ 1 (blank uses the default).
 3. `_ask_date()`: Accept a year ('2021'), year-month ('2021-03') or full date ('2021-03-15').
 4. `_slice_range()`: Trim the data to [start, end]; a None bound means "no limit on that side".